Lab2: Sol SDK Workshop

Edan Chen

edan.chen@ganzin.com.tw

Outline

- Introduction of Sol SDK
- Lab0: Sol SDK Installation
- Lab1: Connect PC to Sol Glasses
- Case Study Demo

Background of Sol SDK

android

- Slow developing process
- Complex environment
- Steep learning curve

vs.

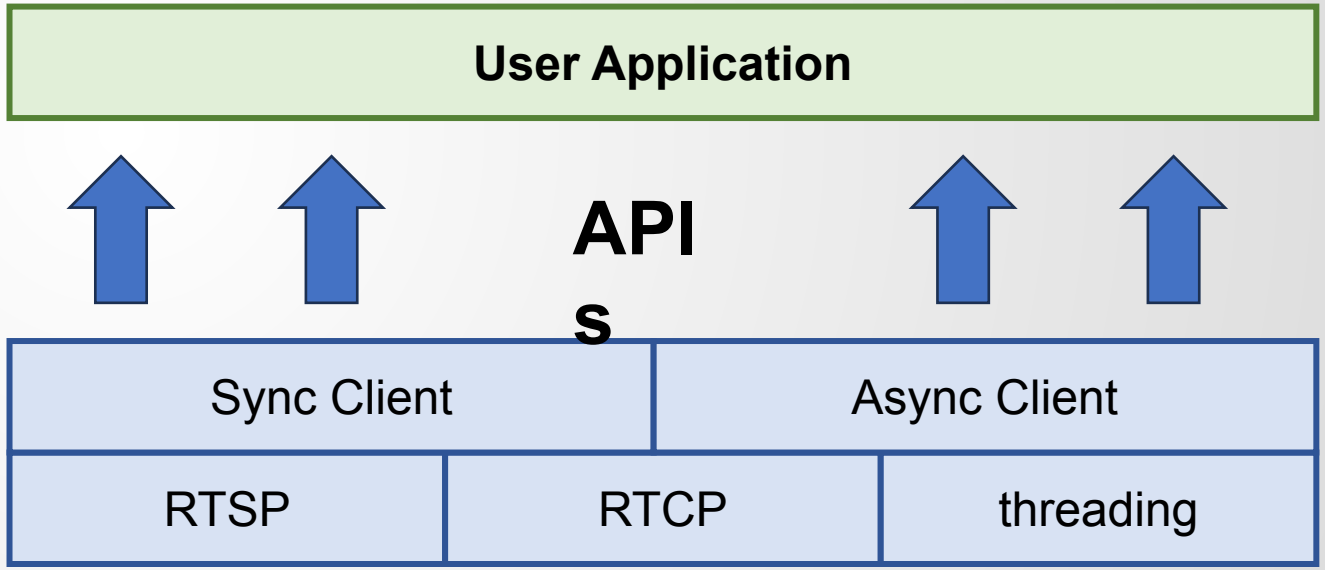
python

- Everyone knows how to use
- TOP programming language in the world!
- Many other packages can be integrated (PsychoPy, OpenAI API, etc.)

Fast eye tracking application development!

Architecture of Sol SDK

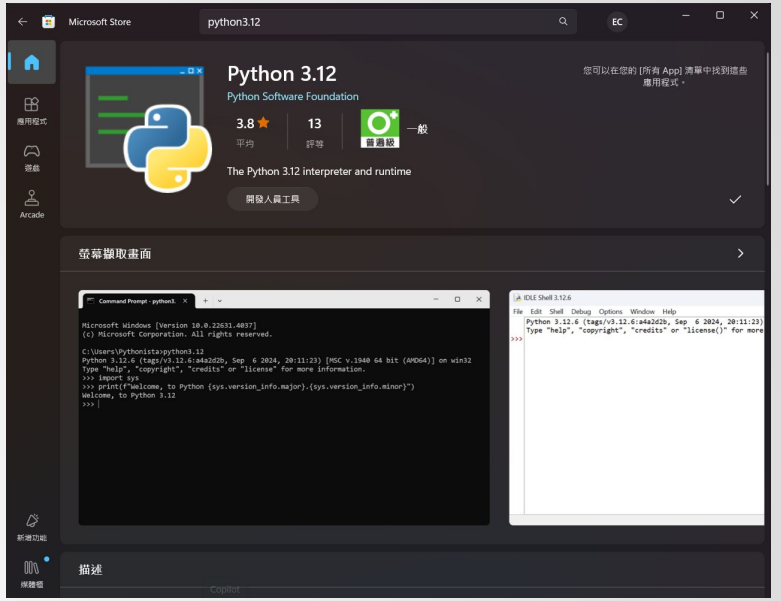
- Communication protocol: RTSP (Real Time Streaming Protocol), RTCP (Real-time Transport Control Protocol)
- Muti-threading: Python threading library
- Provides two kinds of clients
  - Sync Client
    - Easy to program
    - Good start point
    - Poor performance due to blocking
  - Async Client
    - More complicate
    - More real-time demonstration



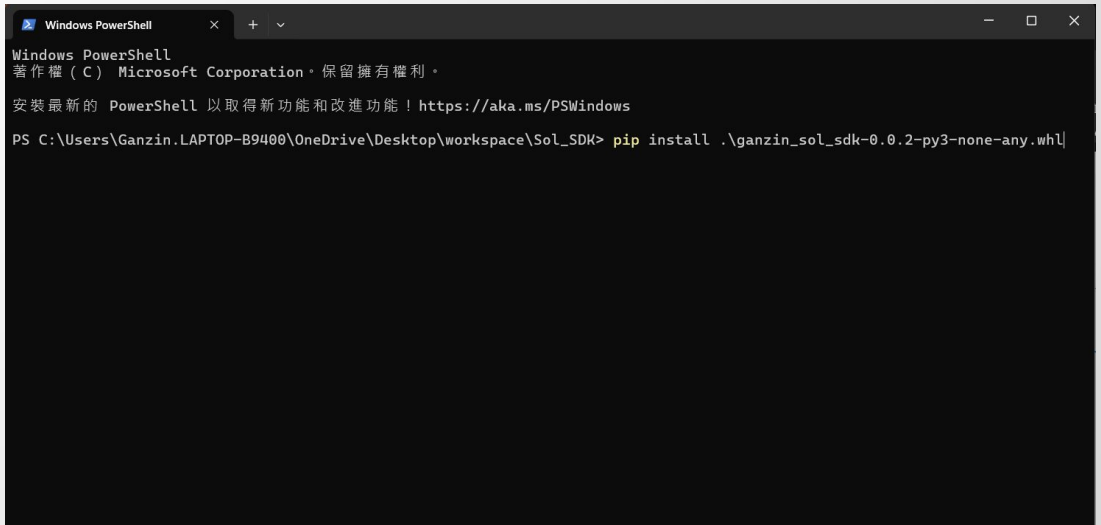
- Let's learn the Sol SDK by going through the example code!!!

Lab0: Sol SDK Installation

Installation

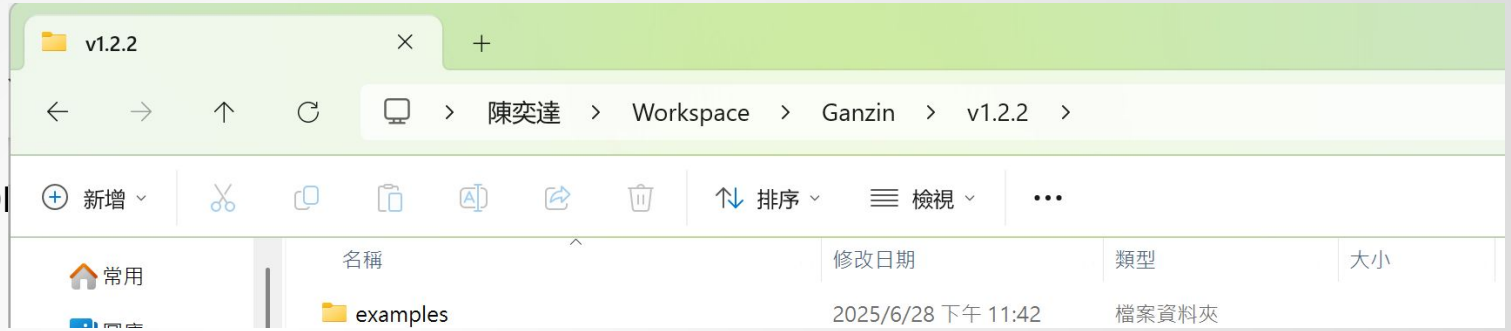


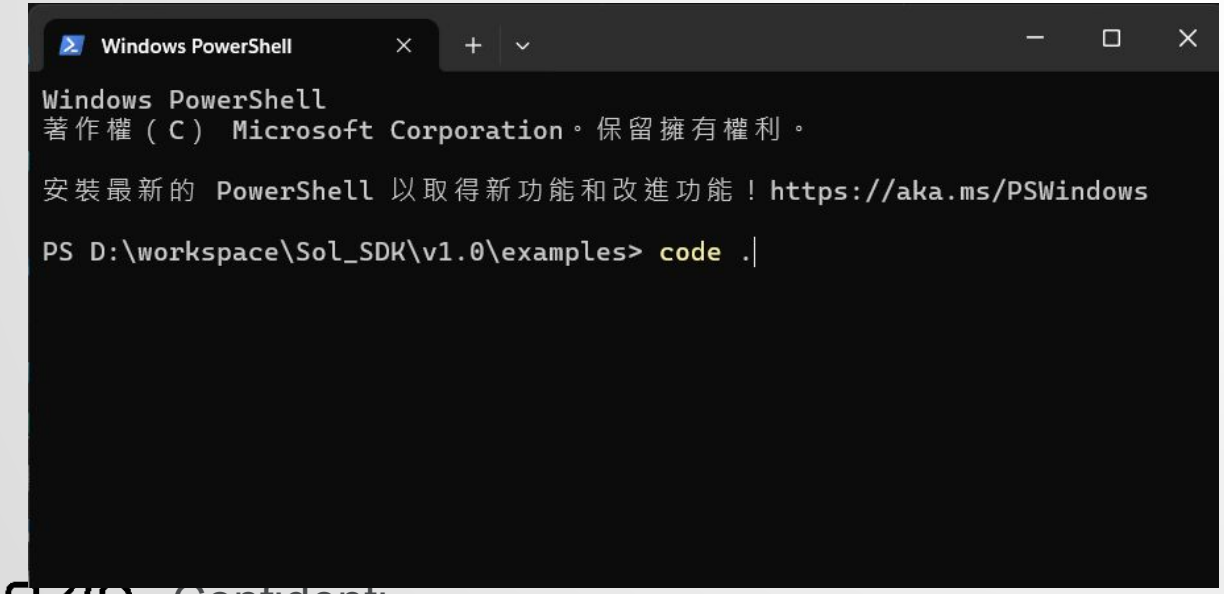
- Install Python3.12
  - win10
    https://www.microsoft.com/store/productId/9NCVDN91XZQP?ocid=pdpshare
  - Mac
    $ brew install python@3.12
- Download Sol SDK (v1.2.2)
  - whl: https://drive.google.com/file/d/1RBcuDe8gsKRCRlWPhi93mX9nXdn4hYgO/view?usp=sharing
  - examples.zip: https://drive.google.com/file/d/1_uN0Wd2OZF26yL34OH5KcF9aq-_qpqrF/view?usp=sharing
  - docs: https://drive.google.com/file/d/1DTEAKElbJQg4UkSn62f5dlCF50cQN1Nb/view?usp=sharing
- Install SDK
  - In the Sol SDK directory
    - Right click to open the powershell (or terminal)
  - *$ python3 -m pip install .\ganzin_sol_sdk-1.0.1-py3-none-any.whl*
- Install other packages will used later
  - *$ python3 -m pip install requests opencv-python pyaudio*



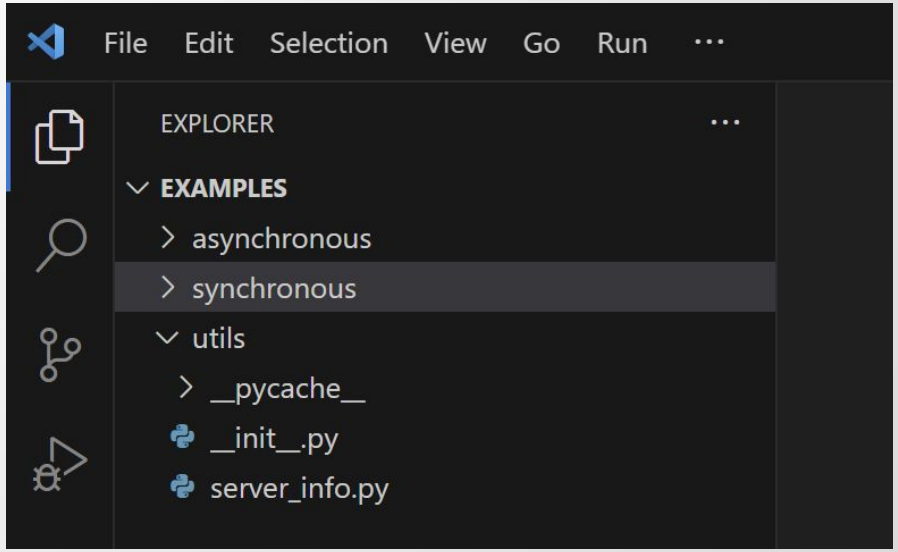
Dive Into Examples

- Extract examples.zip
  - Check the *examples* directory exist
- Open vscode in the examples directo
  - Go into the examples directory
  - Right click and open powershell
  - *$ code .*





- Check the hierarchy in vscode

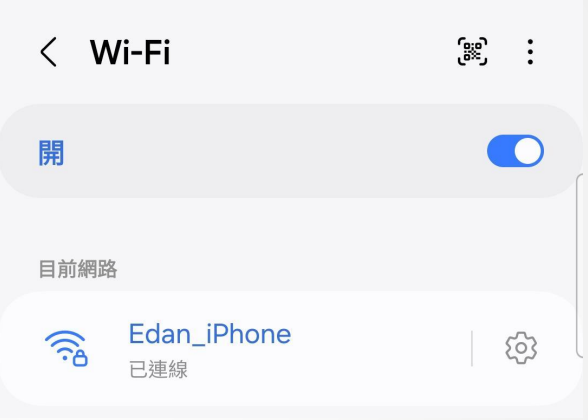


Lab1: Connect PC to Sol Glasses

Previewing the gaze video

Setup Environment (1/2)

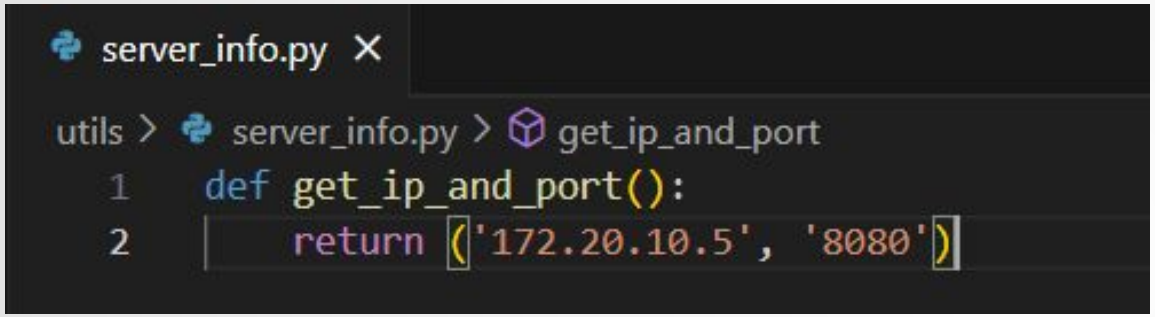
- Connect Sol Glasses's phone and your PC under the same AP (e.g., your own hotspot)



- Open the Android record tool and go into the preview window and click the computer icon
  - Get the IP and Port from the phone screen

Setup Environment (2/2)

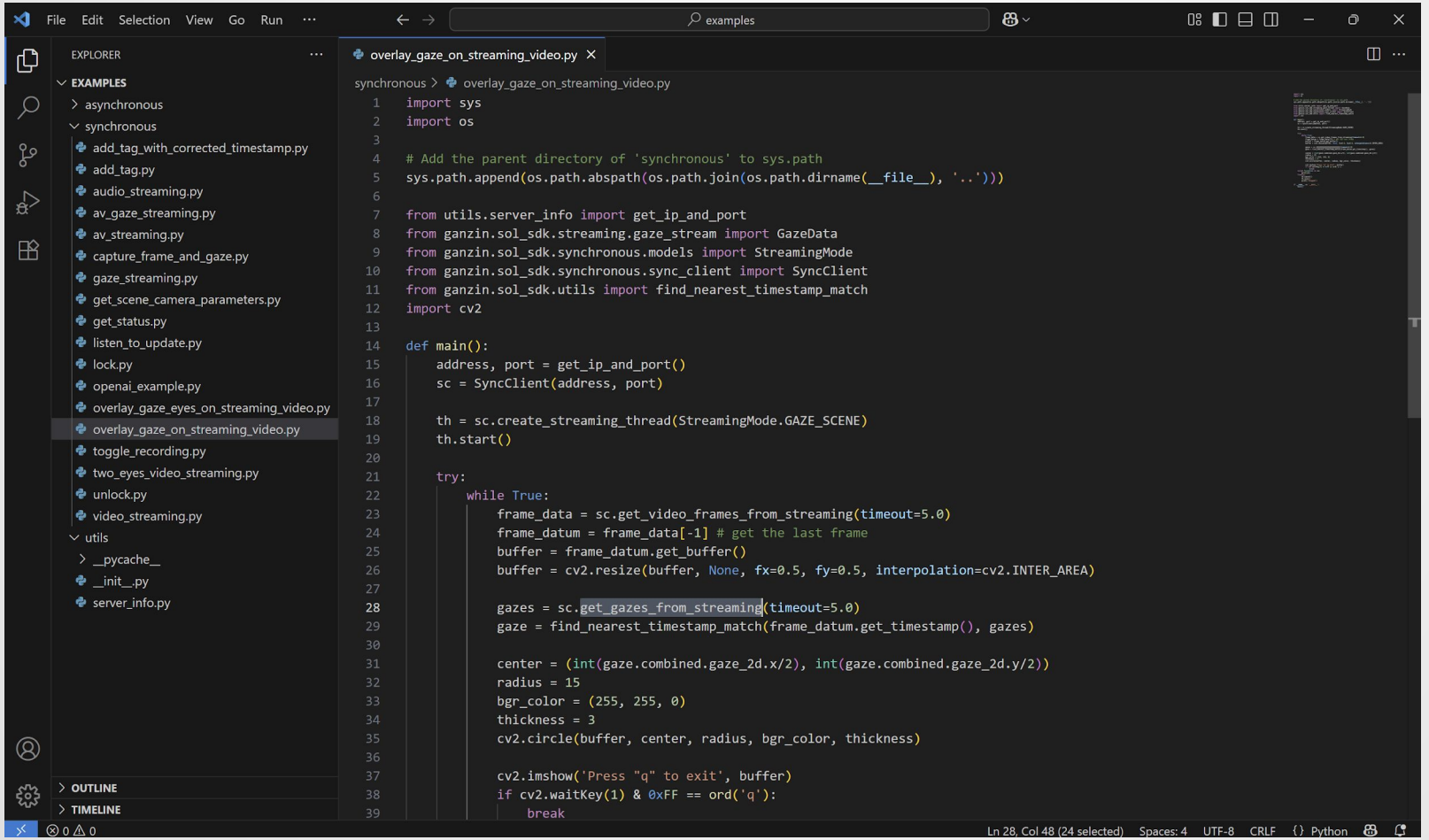
- Fill the ip and port into the *utils/server_info.py*



- Please notice that when using the Sol SDK, the Android record tool needs to stay in the **preview window**

Dive into the Gaze Video Example

- Open the *synchronous/overlay_gaze_on_streaming_video.py* example



Ganzin

Dive into the Code (1/4)

- Import modules
  - sys, os: to make the server_info importable
  - ganzin.sol_sdk: the sdk to remote control the Sol Glasses
  - cv2: visualize the result get from the Sol SDK

```python
import sys
import os

# Add the parent directory of 'synchronous' to sys.path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from utils.server_info import get_ip_and_port
from ganzin.sol_sdk.streaming.gaze_stream import GazeData
from ganzin.sol_sdk.synchronous.models import StreamingMode
from ganzin.sol_sdk.synchronous.sync_client import SyncClient
from ganzin.sol_sdk.utils import find_nearest_timestamp_match
import cv2
```

Dive into the Code (2/4)

- Sync Client setup
  - Get the address and port from *get_ip_and_port()*
  - Create a *SyncClient* using the address and port
  - Create a *streaming thread* for the Sync Client
  - Start the thread

```
14    def main():
15        address, port = get_ip_and_port()
16        sc = SyncClient(address, port)
17
18        th = sc.create_streaming_thread(StreamingMode.GAZE_SCENE)
19        th.start()
```

Dive into the Code (3/4)

- Getting the gazes and the frames
  - Create an infinite while loop to keep getting the gazes and frames
  - Get the frame data from *get_video_frames_from_streaming()*
    - Resize the buffer as the resolution of the Sol Glasses scene camera is 1328x1200 (exceed 1080)
  - Get the gazes data from *get_gazes_from_streaming()*
  - Get the exactly gaze using the timestamp of the frame

```
21      try:
22          while True:
23              frame_data = sc.get_video_frames_from_streaming(timeout=5.0)
24              frame_datum = frame_data[-1] # get the last frame
25              buffer = frame_datum.get_buffer()
26              buffer = cv2.resize(buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
27
28              gazes = sc.get_gazes_from_streaming(timeout=5.0)
29              gaze = find_nearest_timestamp_match(frame_datum.get_timestamp(), gazes)
```

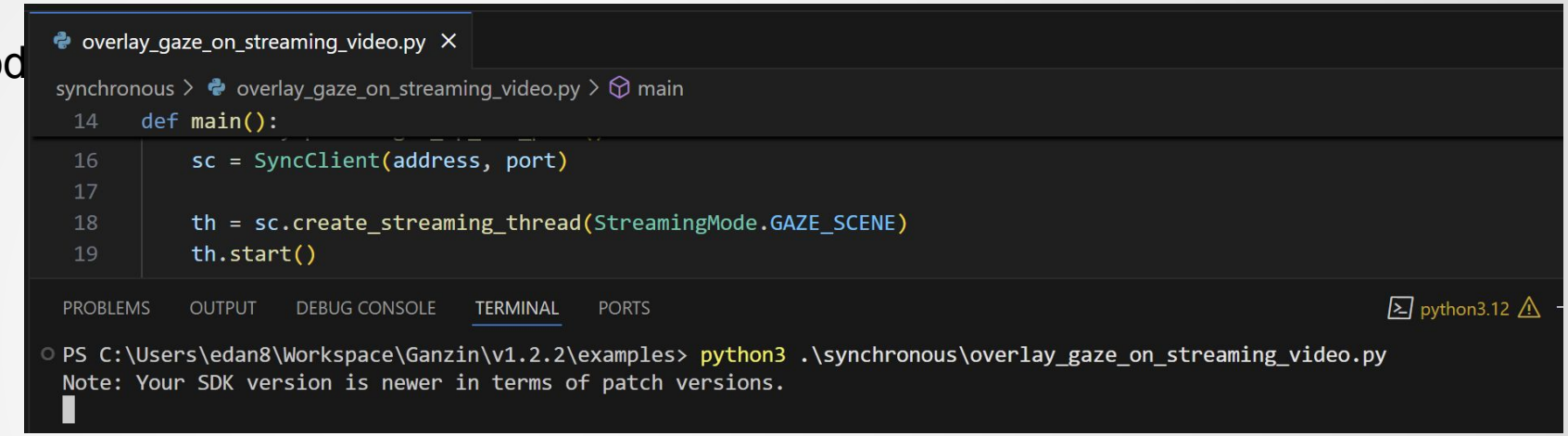
Dive into the Code (4/4)

- Show the result using cv2
  - Calculate the center of the circle
  - Set the circle's radius, color, and thickness
  - Draw the circle using *cv2.circle()*
  - Show the result using *cv2.imshow()*

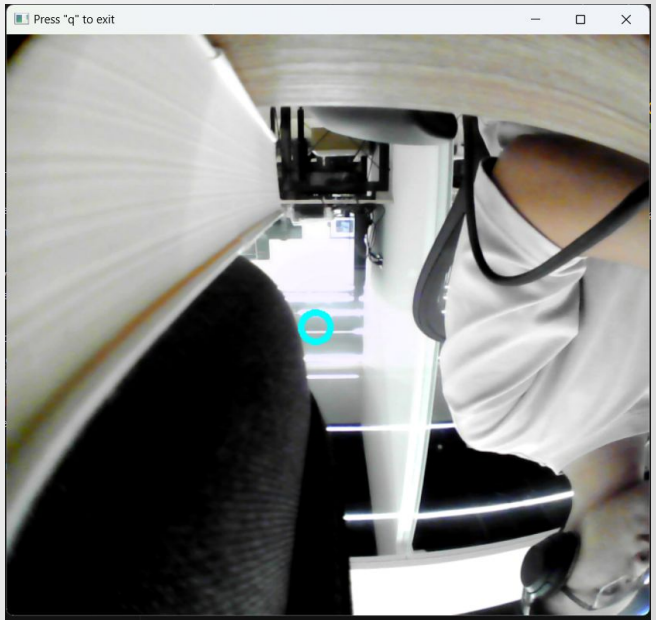
```
31              center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.y/2))
32              radius = 15
33              bgr_color = (255, 255, 0)
34              thickness = 3
35              cv2.circle(buffer, center, radius, bgr_color, thickness)
36
37              cv2.imshow('Press "q" to exit', buffer)
38              if cv2.waitKey(1) & 0xFF == ord('q'):
39                  break
```

Run the Example

- Open terminal from the vscod

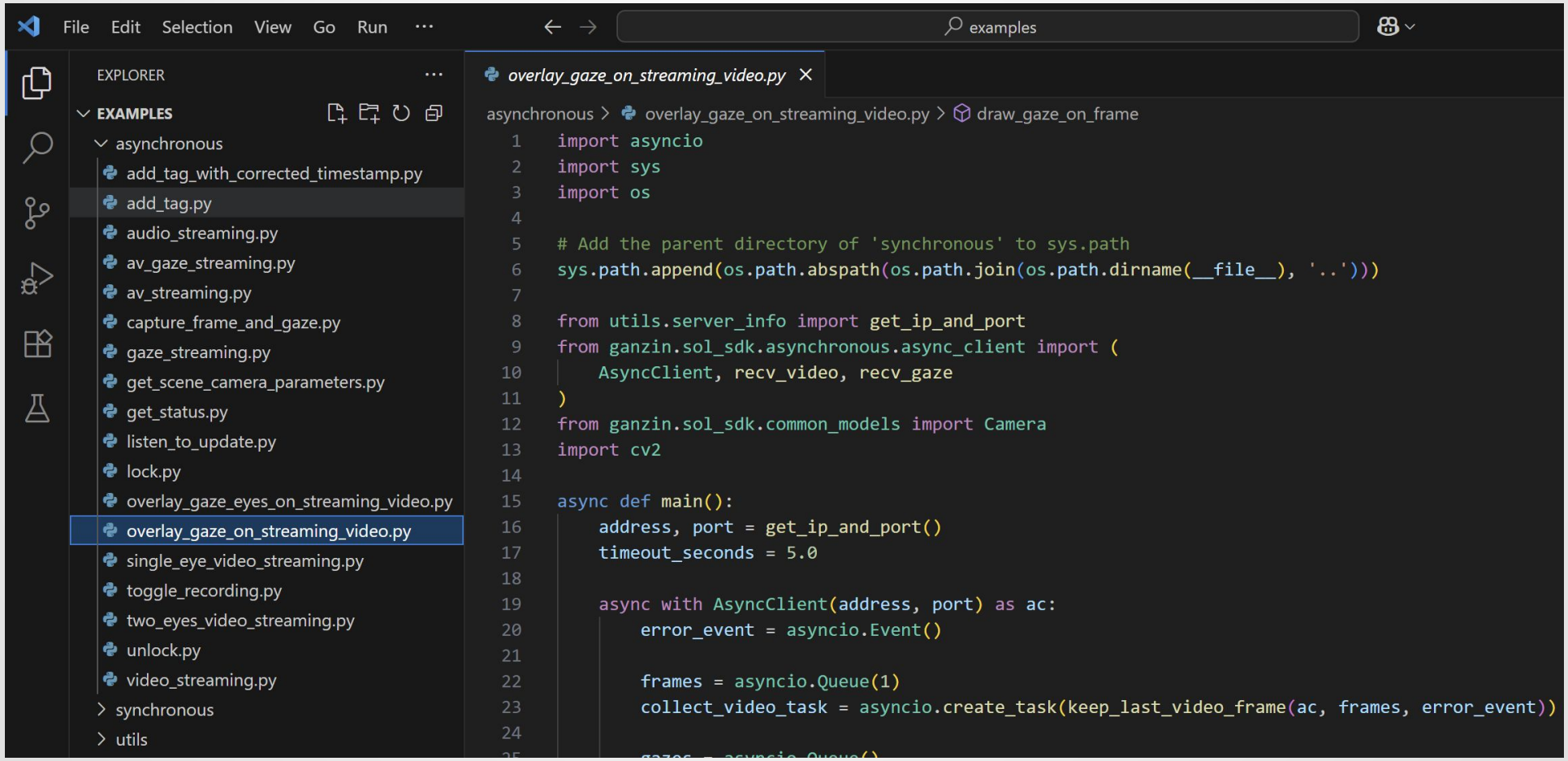


- *$ python3 .\synchronous\overlay_gaze_on_streaming_video.py*
  - Check whether the streaming window appear
  - Press ‘q’ to exit



Dive into the Async Counterpart

- Open the *asynchronous/overlay_gaze_on_streaming_video.py* example



Dive into the Code (1/5)

- Module import
  - asyncio: modules for asynchronous programming, enables the execution of I/O-bound tasks without blocking the execution of other tasks.
  - ganzin.sol_sdk: import the AsyncClient

```python
import asyncio
import sys
import os

# Add the parent directory of 'synchronous' to sys.path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from utils.server_info import get_ip_and_port
from ganzin.sol_sdk.asynchronous.async_client import (
    AsyncClient, recv_video, recv_gaze
)
from ganzin.sol_sdk.common_models import Camera
import cv2
```

Ganzin

Dive into the Code (2/5)

- Main function
  - Set address and port of AsyncClient, timeout_seconds for collect_gaze function (considering internet environment)
  - Create two queues: frames, gazes, for putting the frame/gaze into the queue
  - Create two tasks: collect_video_task, collect_gaze_task, for putting the frame get from the AsyncClient into the queue
  - Await for the *draw_gaze_on_frame()* draw the frame and gaze when there is gaze/frame in the queues

```python
async def main():
    address, port = get_ip_and_port()
    timeout_seconds = 5.0

    async with AsyncClient(address, port) as ac:
        error_event = asyncio.Event()

        frames = asyncio.Queue(1)
        collect_video_task = asyncio.create_task(keep_last_video_frame(ac, frames, error_event))

        gazes = asyncio.Queue()
        collect_gaze_task = asyncio.create_task(collect_gaze(ac, gazes, error_event, timeout_seconds))

        try:
            await draw_gaze_on_frame(frames, gazes, error_event, timeout_seconds)
        finally:
            collect_video_task.cancel()
            collect_gaze_task.cancel()
```

Dive into the Code (3/5)

- *keep_last_video_frame()*
  - async for loop for getting frame from *recv_video()*
    - This will be blocked by the IO
  - Remove the item in queue if full ASAP, the queue has only one seat
  - Put the frame into the queue ASAP
- *collect_gaze()*
  - async for loop for getting gaze from *rec_gaze()*
  - put the gaze into the queue
    - The queue can preserve multiple gazes

```python
async def keep_last_video_frame(ac: AsyncClient, queue: asyncio.Queue, error_event: asyncio.Event) -> None:
    async for frame in recv_video(ac, Camera.SCENE):
        if error_event.is_set():
            break

        if queue.full():
            queue.get_nowait()
        queue.put_nowait(frame)

async def collect_gaze(ac: AsyncClient, queue: asyncio.Queue, error_event: asyncio.Event, timeout) -> None:
    try:
        async for gaze in recv_gaze(ac):
            if error_event.is_set():
                break

            await asyncio.wait_for(queue.put(gaze), timeout=timeout)
    except Exception as e:
        error_event.set()
```

Dive into the Code (4/5)

- *draw_gaze_on_frame()*
  - Get the frame from the *frame_queue*
  - Get the gaze from the *gaze_queue*
  - Resize frame buffer and draw the gaze on the frame using cv2

```
53  async def draw_gaze_on_frame(frame_queue, gazes, error_event: asyncio.Event, timeout):
54      while not error_event.is_set():
55          frame = await get_video_frame(frame_queue, timeout)
56          gaze = await find_gaze_near_frame(gazes, frame.get_timestamp(), timeout)
57          frame_buffer = frame.get_buffer()
58          frame_buffer = cv2.resize(frame_buffer, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)
59
60          center = (int(gaze.combined.gaze_2d.x/2), int(gaze.combined.gaze_2d.y/2))
61          radius = 15
62          bgr_color = (255, 255, 0)
63          thickness = 3
64          cv2.circle(frame_buffer, center, radius, bgr_color, thickness)
65
66          cv2.imshow('Press "q" to exit', frame_buffer)
67          if cv2.waitKey(1) & 0xFF == ord('q'):
68              return
```

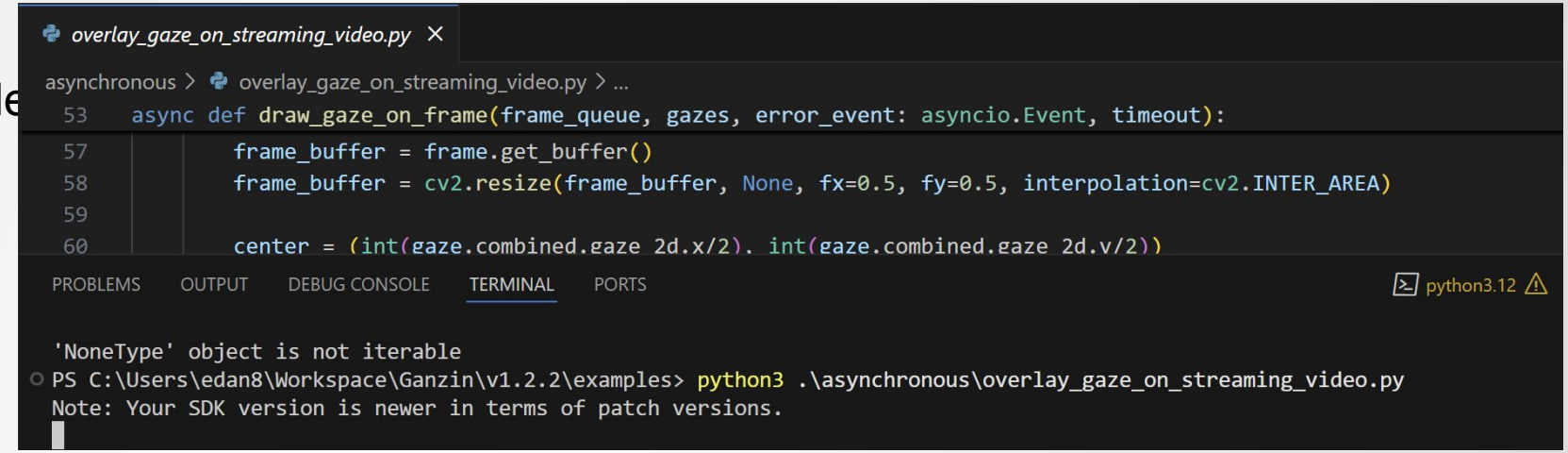
Dive into the Code (5/5)

- *get_video_frame()*
  - Async function for getting the frame out of the queue
- *find_gaze_near_frame()*
  - Async function for finding the most appropriate gaze in the gaze queue

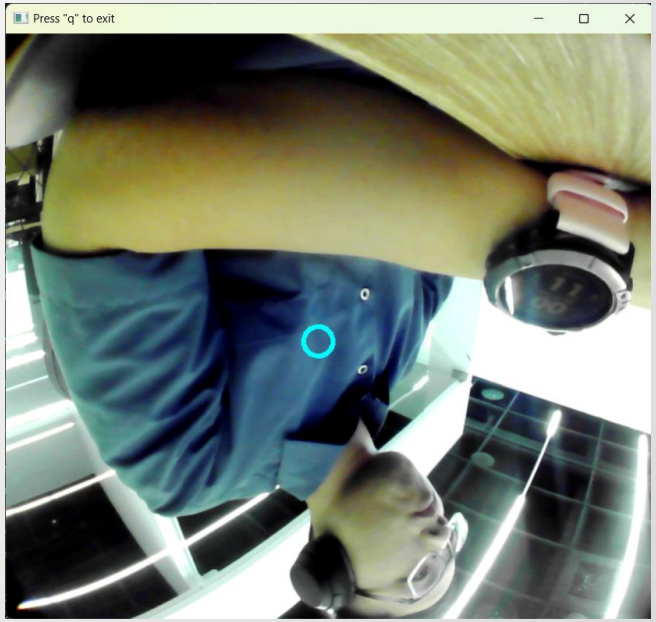
```
70  async def get_video_frame(queue, timeout):
71      return await asyncio.wait_for(queue.get(), timeout=timeout)
72
73  async def find_gaze_near_frame(queue, timestamp, timeout):
74      item = await asyncio.wait_for(queue.get(), timeout=timeout)
75      if item.get_timestamp() > timestamp:
76          return item
77
78      while True:
79          if queue.empty():
80              return item
81          else:
82              next_item = queue.get_nowait()
83              if next_item.get_timestamp() > timestamp:
84                  return next_item
85              item = next_item
```

Run the Example

- Open terminal from the vscode

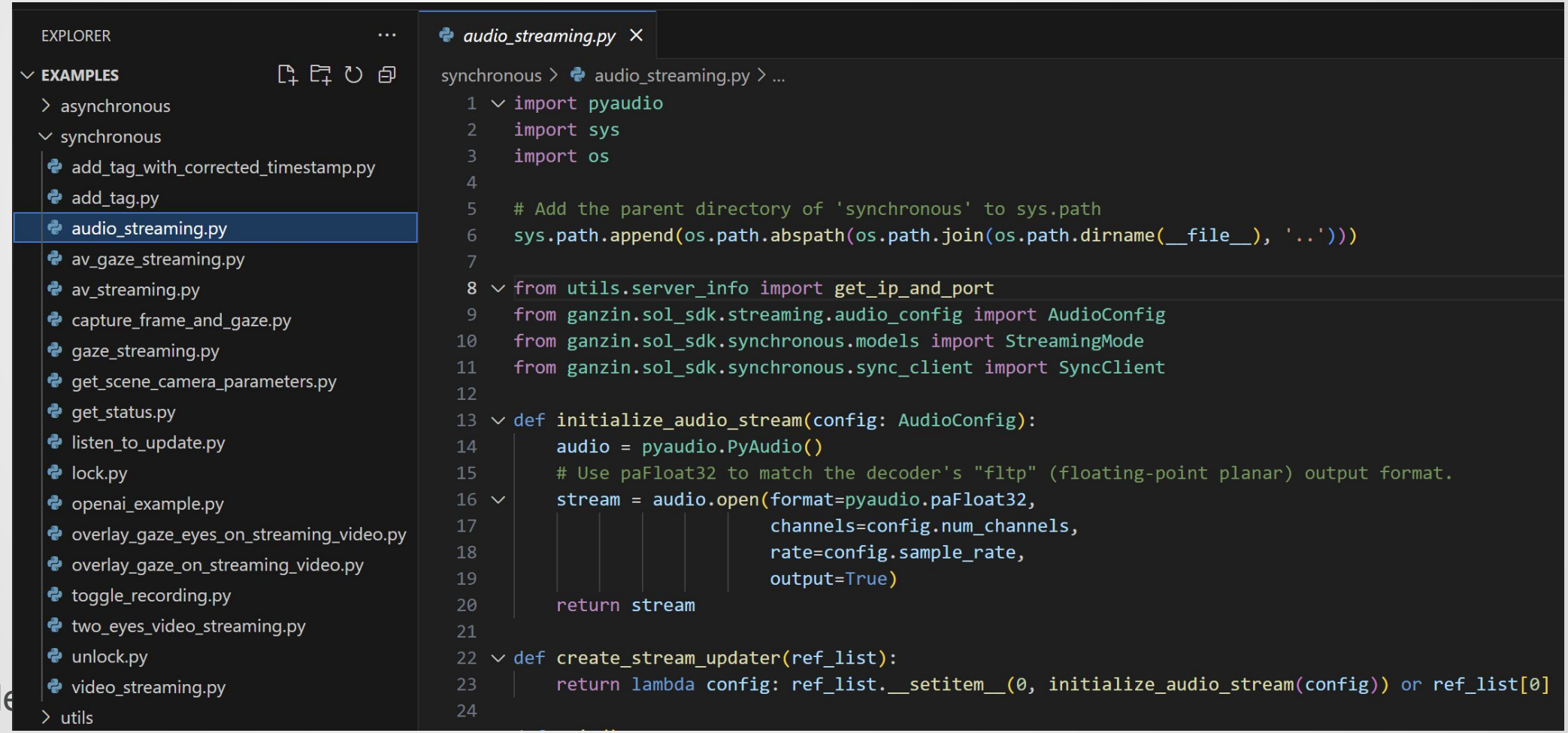


- *$ python3 .\asynchronous\overlay_gaze_on_streaming_video.py*
  - Check whether the streaming window appear
  - Press ‘q’ to exit
  - Same result from the synchronous counterpart



Dive into the Audio Example

- Open the *synchronous/audio_streaming.py* example



Dive into the Code (1/3)

- Module import
  - pyaudio: modules to process the audio get from Sol Glasses
  - ganzin.sol_sdk: import the SyncClient and AudioConfig

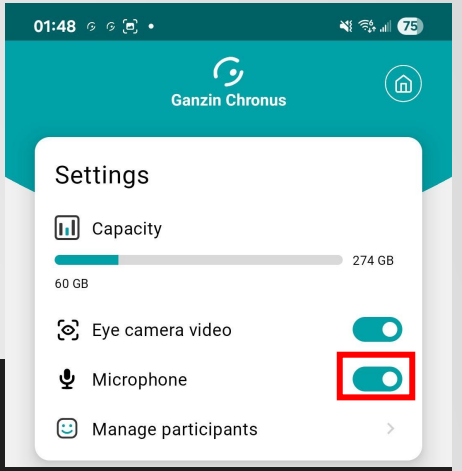
```python
import pyaudio
import sys
import os

# Add the parent directory of 'synchronous' to sys.path
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

from utils.server_info import get_ip_and_port
from ganzin.sol_sdk.streaming.audio_config import AudioConfig
from ganzin.sol_sdk.synchronous.models import StreamingMode
from ganzin.sol_sdk.synchronous.sync_client import SyncClient
```

Dive into the Code (2/3)

- Main function
  - Set address and port of AsyncClient
  - Check the audio_enabled label
    - Need to enable audio in Chronus
  - Create a steam to get the audio data
  - Setup the audio callback function
    - Initialize audio stream when new audio config is receiv
  - Create steaming thread for pure audio
  - In while loop
    - Get audio frame
    - Write the pcm of the frame to the audio stream and play it out with PyAudio




```python
def main():
    address, port = get_ip_and_port()
    sc = SyncClient(address, port)
    if not sc.get_status().audio_enabled:
        print("Warning: Please enable 'Microphone' in the server settings.")
        return

    streams = [None]

    # Create a callback function that initializes the audio stream when
    # new audio configuration is received. This function updates the stream
    # reference in the streams list.
    update_audio_stream = create_stream_updater(streams)

    print("Audio playback started. Press Ctrl+C to stop.")

    th = sc.create_streaming_thread(StreamingMode.AUDIO, update_audio_stream)
    th.start()

    try:
        while True:
            frame_data = sc.get_audio_frames_from_streaming(timeout=5.0)
            if not frame_data:
                continue
            for sample in frame_data:
                streams[0].write(sample.get_pcm())

    except KeyboardInterrupt: # Press Ctrl-C to stop
        pass
```

Dive into the Code (3/3)

- *initialize_audio_stream()*
  - Given an *AudioConfig* (sample rate, channels), it:
    - Instantiates a *pyaudio.PyAudio()* object
    - Opens a stream with paFloat32 format matching the decoder's "fltp"
    - Returns the ready-to-write PyAudio stream
- *create_steam_updater()*
  - Your helper that takes a one-element list (*streams*) and returns a callback fn (config: *AudioConfig*) →

```
13  def initialize_audio_stream(config: AudioConfig):
14      audio = pyaudio.PyAudio()
15      # Use paFloat32 to match the decoder's "fltp" (floating-point planar) output format.
16      stream = audio.open(format=pyaudio.paFloat32,
17                          channels=config.num_channels,
18                          rate=config.sample_rate,
19                          output=True)
20      return stream
21
22  def create_stream_updater(ref_list):
23      return lambda config: ref_list.__setitem__(0, initialize_audio_stream(config)) or ref_list[0]
```

Run the Example

- Open terminal from the vscode

- *$ python3 .\asynchronous\audio_streaming.py*
  - Check whether you can hear the sound recorded by Sol's microphone
  - Press 'ctrl-c' to exit

Case Study Demo

Q&A